

CS 3300 - Tools of the Trade #2

Demo Notes

VS Code + Maven + JUnit 5 + Debugging

Follow-along notes for the Maven, JUnit 5, and debugging demo

0. Demo environment and versions

Use these versions for the class demo. The Maven quickstart archetype may generate older defaults; we will update the generated project deliberately.

Component	Course demo configuration
JDK	21
Maven	3.9.x

1. Before the demo: verify your setup

Open VS Code and a terminal. Verify:

```
java --version
mvn --version
```

- JDK 21 is installed on the machine.
- Maven 3.9.x is installed on the machine and available on PATH.
- VS Code is installed.
- Microsoft Extension Pack for Java is installed in VS Code. It includes Language Support for Java, Debugger for Java, Test Runner for Java, Maven for Java, and related tools.

2. Create a Maven project in VS Code

1. Open the Command Palette (Ctrl+Shift+P on Windows/Linux; Cmd+Shift+P on macOS).
2. Search for Maven and choose Maven: Create Maven Project.
3. Choose maven-archetype-quickstart.
4. Choose the offered archetype version.
5. Enter a groupId, for example: edu.cs3300
6. Enter an artifactId, for example: mavendemo
7. Accept or enter the project version, for example: 1 or 1.0-SNAPSHOT.
8. Choose a location and open the generated project folder in VS Code.

Maven terminology

Term	Meaning
Archetype	A Maven project template. quickstart creates a basic Java project with standard Maven folders.
Archetype version	Version of the project template; it is not the Java version or your project version.
groupId	Identifies the organization/group. Often uses reverse-domain naming, e.g.,

	edu.cs3300 or com.example.
artifactId	Identifies this specific project within the group; often becomes the project/artifact name.
version	Version of your project.
SNAPSHOT	Convention indicating a development/in-progress project version.
package	Base Java package for generated code; often derived from the groupId.

3. Inspect the generated Maven project

```
demo/
|-- pom.xml
`-- src/
    |-- main/java/com/example/App.java
    `-- test/java/com/example/AppTest.java
```

- src/main/java - production/application code
- src/test/java - automated test code
- pom.xml - project metadata, dependencies, compiler/build configuration, and plugins

```
dir          # Windows PowerShell: verify pom.xml is visible
mvn test
```

4. Update the generated pom.xml for this course

The quickstart archetype may generate JUnit 4 and older plugin/compiler defaults. Explain that an archetype is a starting point; we configure the project for the versions used in this course.

Use this simplified known-good POM

```
<properties>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  <maven.compiler.release>21</maven.compiler.release>
</properties>

<dependencies>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.11.2</version>
    <scope>test</scope>
  </dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.14.1</version>
    </plugin>
    <plugin>
      <artifactId>maven-surefire-plugin</artifactId>
      <version>3.6.0</version>
    </plugin>
  </plugins>
```

```
</build>
</project>
```

- `maven.compiler.release = 21`: compile against the Java 21 API and produce Java 21-compatible bytecode.
- `org.junit.jupiter:junit-jupiter:5.11.2`: the JUnit 5 dependency used by our test source code.
- `scope = test`: JUnit is needed for tests, not as a runtime dependency of the application.
- `maven-compiler-plugin`: compiles Java source/test code.
- `maven-surefire-plugin`: discovers and runs tests during the Maven test phase.

5. Remove the generated JUnit 4 test

The quickstart archetype may generate `AppTest.java` using JUnit 4 imports such as `org.junit.Test` and `org.junit.Assert`. Because the project is now configured for JUnit 5, delete the generated `AppTest.java` for this demo.

```
src/test/java/com/example/AppTest.java  <-- delete
```

6. Verify Maven before writing the demo code

```
mvn clean
mvn test
```

What Maven is doing:

- Reading `pom.xml`.
- Resolving required plugins and dependencies.
- Downloading missing artifacts from Maven Central on first use.
- Caching downloaded artifacts in Maven's local repository for later builds.
- Compiling production and test code, then running tests through Surefire.

7. Create `ShippingCalculator.java`

Create `src/main/java/com/example/ShippingCalculator.java`:

```
package com.example;

public class ShippingCalculator {
    public double calculateShipping(double orderTotal, boolean member) {
        if (orderTotal >= 50) {
            return 0.0;
        }
        if (member) {
            return 4.99;
        }
        return 8.99;
    }
}
```

8. Identify what should be tested

Before writing tests, consider: "What inputs would you test?" Use these cases:

Case	Input	Expected
Normal non-member	\$30, false	\$8.99
Member	\$30, true	\$4.99
Free shipping	\$75, false	\$0.00
Boundary	\$50, false	\$0.00
Just below boundary	\$49.99, false	\$8.99

9. Create the first JUnit 5 test: Arrange - Act - Assert

Create src/test/java/com/example/ShippingCalculatorTest.java:

```
package com.example;

import static org.junit.jupiter.api.Assertions.assertEquals;
import org.junit.jupiter.api.Test;

public class ShippingCalculatorTest {
    @Test
    void nonMemberPaysStandardShipping() {
        // Arrange
        ShippingCalculator calculator = new ShippingCalculator();

        // Act
        double shipping = calculator.calculateShipping(30.00, false);

        // Assert
        assertEquals(8.99, shipping, 0.001);
    }
}
```

The third assertEquals argument

```
assertEquals(8.99, shipping, 0.001);
```

- 8.99 = expected value
- shipping = actual value
- 0.001 = allowed delta/tolerance for floating-point comparison

Conceptually, JUnit accepts the comparison when the actual value is sufficiently close to the expected value within the specified tolerance. This avoids relying on exact equality for double values.

10. Run the test in two ways

A. VS Code Testing UI

Use the test gutter/status icon or Testing view to run the test. Depending on VS Code configuration, inline Run Test / Debug Test CodeLens text may or may not appear.

B. Maven

```
mvn test
```

mvn test works independently of VS Code's test buttons, which is one of the benefits of having the build/test process defined by the project instead of the IDE. Point out the summary: tests run, failures, errors, skipped, and BUILD SUCCESS.

11. Add tests for branches and boundaries

```
@Test
void memberGetsReducedShipping() {
    ShippingCalculator calculator = new ShippingCalculator();
    double shipping = calculator.calculateShipping(30.00, true);
    assertEquals(4.99, shipping, 0.001);
}

@Test
void orderOverThresholdGetsFreeShipping() {
    ShippingCalculator calculator = new ShippingCalculator();
    double shipping = calculator.calculateShipping(75.00, false);
    assertEquals(0.0, shipping, 0.001);
}

@Test
void orderAtThresholdGetsFreeShipping() {
    ShippingCalculator calculator = new ShippingCalculator();
    double shipping = calculator.calculateShipping(50.00, false);
    assertEquals(0.0, shipping, 0.001);
}

@Test
void orderBelowThresholdPaysShipping() {
    ShippingCalculator calculator = new ShippingCalculator();
    double shipping = calculator.calculateShipping(49.99, false);
    assertEquals(8.99, shipping, 0.001);
}

mvn test
```

12. Create a realistic boundary defect

Temporarily change the implementation from:

```
if (orderTotal >= 50) {
```

to:

```
if (orderTotal > 50) {    // intentional bug
```

```
mvn test
```

Expected outcome: most tests still pass, but orderAtThresholdGetsFreeShipping fails. Compare expected vs. actual. One passing test does not prove the method is correct.

13. Debugging with a regular breakpoint

Before debugging

- Verify Microsoft Debugger for Java is installed/enabled (normally included in Extension Pack for Java).
- If Debug Test reports "Cannot find debug action!", verify Debugger for Java and Test Runner for Java are enabled. If conflicting Java extensions are installed, disable the conflicting extension and reload VS Code.
- A launch.json should not normally be necessary just to debug a JUnit test through the Java test integration.

Regular breakpoint

9. Open ShippingCalculator.java.
10. Click in the gutter beside the condition to create a red breakpoint dot.

```
if (orderTotal > 50) {
```

11. Open the Testing view, locate orderAtThresholdGetsFreeShipping, right-click it, and choose Debug Test. Some VS Code configurations also show Debug Test directly in the editor.
12. When execution pauses, inspect Variables: orderTotal = 50.0 and member = false.
13. Use Step Over to follow the branch. Observe that 50 > 50 is false, so execution skips free shipping.
14. Fix the condition back to >= and rerun the full test suite.

Core debugger controls

Control	Meaning
Continue	Resume until the next breakpoint or program completion.
Step Over	Execute the current line without entering a called method.
Step Into	Enter a method called by the current line.
Step Out	Finish the current method and return to its caller.

14. Demonstrate all main breakpoint types

A. Regular breakpoint

Use this when you want to answer: "When does execution reach this line?"

```
if (orderTotal >= 50) {    // regular breakpoint here
```

B. Conditional breakpoint

Right-click the breakpoint -> Edit Breakpoint -> Expression. Use:

```
orderTotal == 50.0
```

Use this when you want to answer: "When does execution reach this line under this particular condition?" This is useful when several tests call the same method but you only want to pause for the \$50 case.

C. Logpoint

Right-click in the gutter -> Add Logpoint. Use a message such as:

```
Shipping calculation: total={orderTotal}, member={member}
```

A logpoint writes diagnostic information to the Debug Console without stopping execution and without adding `System.out.println` statements to source code.

D. Data breakpoint

Data breakpoints pause when a supported field changes. Add a small field for demonstration:

```
private int calculationsPerformed = 0;

public double calculateShipping(double orderTotal, boolean member) {
    calculationsPerformed++;
    // existing shipping logic...
}
```

15. Start a debug session and pause first with a regular breakpoint.
16. In Variables, locate `calculationsPerformed`.
17. If the Java debugger offers Break on Value Change / Data Breakpoint for the field, enable it.
18. Continue execution and observe the debugger pause when `calculationsPerformed` changes.

Breakpoint type	What it is useful for
Regular	Pause whenever execution reaches a line.
Conditional	Pause at a line only when an expression is true.
Logpoint	Log runtime information without pausing or editing source code.
Data	Pause when a supported field changes value.

15. Refactor repeated setup with @BeforeEach

Once several tests repeat the same calculator construction, introduce common setup:

```
import org.junit.jupiter.api.BeforeEach;

private ShippingCalculator calculator;

@BeforeEach
void setUp() {
    calculator = new ShippingCalculator();
}
```

Then tests can use the field directly. Explain that `@BeforeEach` runs before each test, helping each test start with a fresh setup and remain independent.

16. Maven troubleshooting: normal output, -e, and -X

When Maven fails, it often explicitly suggests `-e` and `-X`. Use this escalation path: `mvn test`

- Start with the normal error. It is often enough.

```
mvn test -e
```

- `-e` shows more error/exception stack-trace detail.

```
mvn test -X
```

- `-X` enables very verbose Maven debug output: plugin configuration, dependency resolution, execution details, and more.